

[PORTADA OFICIAL IES ÁGORA]

Guardar la portada oficial como docs/portada-oficial.pdf

Resumen

Komorebi Café es una plataforma web de gestión integral diseñada para una cadena ficticia de catorce cafés temáticos japoneses. El nombre del proyecto proviene del término japonés *komorebi* (木漏れ日), que describe la luz solar filtrada entre las hojas de los árboles, evocando la atmósfera serena y cuidada que caracteriza estos espacios.

El sistema cubre la totalidad del ciclo operativo: gestión de reservas con control de idempotencia, carta digital con filtros por alérgeno, programa de fidelización, panel de cocina en tiempo real (Kitchen Display System (KDS)), control de salud de animales del local, operaciones de recepción con check-in/checkout y generación asíncrona de facturas en PDF.

Desde el punto de vista técnico, la solución está construida sobre PHP 8.4 con un framework Model-View-Controller (MVC) propio (sin Laravel ni Symfony como capa de aplicación), siguiendo los estándares PHP Standards Recommendation (PSR)-7, PSR-14 y PSR-15. La arquitectura sigue los principios 12-Factor App, con despliegue mediante Docker Compose y Railway. El sistema incluye más de 250 rutas, 53 servicios de negocio, 29 repositorios y más de 95 endpoints de Application Programming Interface (API) Representational State Transfer (REST).

Palabras clave: PHP 8.4, MVC, Docker, PSR-7, PSR-15, Result pattern, reservas, fidelización, cafés temáticos, 12-Factor.

Abstract

Komorebi Café is a comprehensive web management platform designed for a fictional chain of fourteen Japanese-themed cafés. The project name derives from the Japanese term *komorebi* (木漏れ日), describing sunlight filtering through tree leaves, evoking the serene and carefully crafted atmosphere that defines these spaces.

The system covers the complete operational cycle: reservation management with idempotency control, a digital menu with allergen filters, a loyalty programme, a real-time Kitchen Display System (KDS), animal health tracking, reception operations with check-in/checkout, and asynchronous PDF invoice generation.

Technically, the solution is built on PHP 8.4 with a custom MVC framework (no Laravel/Symfony application layer), following PSR-7, PSR-14 and PSR-15 standards. The architecture adheres to the 12-Factor App principles, deployed via Docker Compose and Railway. The system comprises over 250 routes, 53 business services, 29 repositories and more than 95 REST API endpoints.

Keywords: PHP 8.4, MVC, Docker, PSR-7, PSR-15, Result pattern, reservations, loyalty, themed cafés, 12-Factor.

Índice general

Índice de figuras

Índice de tablas

Capítulo 1

Introducción General

1.1. Origen y concepto del proyecto

El punto de partida de Komorebi Café es la pregunta: ¿puede una aplicación web de gestión ser, al mismo tiempo, un reflejo fiel de la identidad de la marca que sirve? El proyecto nace de la voluntad de demostrar que la tecnología y la estética no son opuestos: un sistema de software robusto puede encarnar la filosofía tranquila, detallista y contemplativa que inspira el concepto de *komorebi*.

La palabra japonesa *komorebi* (木漏れ日) se compone de tres kanjis:

- 木 (*ki*) —árbol
- 漏れ (*more*) —filtrar, escapar
- 日 (*bi*) —luz, sol

Juntos describen ese fenómeno visual fugaz en que la luz solar se cuela entre las hojas creando patrones de clarooscuro. Es una palabra sin traducción exacta al español, pero precisamente esa intraducibilidad la convierte en símbolo perfecto: algunas experiencias solo se pueden *vivir*, no describir.

1.2. Los catorce cafés temáticos

La cadena ficticia está compuesta por catorce cafés, cada uno inspirado en un universo estético japonés diferente. La ?? recoge los establecimientos registrados en el sistema.

Tabla 1.1: Catálogo de cafés temáticos de Komorebi Café

Slug	Nombre	Animal / Categoría
neko-no-niwa	Neko no Niwa (猫の庭)	Gato (lounge)
usagi-paradise	Usagi Paradise (うさぎパラダイス)	Conejo (lounge)
soft-cloud	Soft Cloud (ソフトクラウド)	Chinchilla (lounge)
chipmunk-forest	Chipmunk Forest (シマリスの森)	Ardilla (lounge)

(continuación de la ??)

Slug	Nombre	Animal / Categoría
mame-shiba-cafe	Mame Shiba Café (豆柴カフェ)	Perro Shiba Inu (playroom)
mipig-cafe	Mipig Cafe (マイピッグカフェ)	Cerdito miniatura (playroom)
parrot-talk	Parrot Talk (おしゃべりオウム)	Loro (playroom)
capyba-land	Capyba Land (カピバランド)	Capybara (farm)
alpaca-hill	Alpaca Hill (アルパカの丘)	Alpaca (farm)
little-hooves	Little Hooves (小さなひづめ)	Caballo miniatura (farm)
quack-club	Quack Club (クワッククラブ)	Pato (farm)
pui-pui-house	Pui Pui House (プイプイハウス)	Cobaya (zen)
prairie-town	Prairie Town (プレーリータウン)	Perrito de pradera (zen)
slow-life	Slow Life (スローライフ)	Tortuga (zen)

1.3. Misión, visión y valores

Misión

Ofrecer a cada cliente una experiencia de visita única, combinando una carta artesanal con un entorno temático cuidado y un servicio personalizado habilitado por tecnología.

Visión

Ser la referencia en España de la hostelería temática de inspiración japonesa, reconocidos tanto por la calidad de nuestros productos como por la excelencia tecnológica de nuestros sistemas de gestión.

Valores

Ma (間, el espacio entre las cosas), *omotenashi* (hospitalidad total), sostenibilidad, belleza en los detalles y apertura cultural.

1.4. Alcance y finalidad del proyecto

El proyecto abarca el diseño, implementación, pruebas y documentación de una plataforma web completa que gestiona:

1. La presencia pública de la cadena (carta, reservas en línea, historia, FAQ)
2. La autenticación y gestión de cuentas de usuario

3. El programa de fidelización con tarjeta digital y canjes
4. El backoffice de administración (usuarios, cafés, menú, reseñas, animales)
5. Los paneles de operación: recepción, cocina (KDS), manager y keeper
6. La API REST para integración con clientes móviles y terceros

1.5. Stack tecnológico

La ?? resume las tecnologías principales utilizadas en el proyecto.

Tabla 1.2: Stack tecnológico del proyecto

	Capa	Tecnología	Versión / Detalles
Lenguaje backend	PHP		8.4 (strict types en todos los archivos)
Framework	MVC custom		Sin Laravel / Symfony, PSR-7/14/15
Servidor HTTP	FrankenPHP		Caddy + PHP integrado
Base de datos	MySQL		8.0, 19 migraciones SQL
Caché / Cola	Redis		7.x, sesiones + cola de trabajos
Frontend	Alpine.js		3.x, sin bundler en producción
CSS	CSS3 custom		Variables, Grid, Flexbox
Contenedores	Docker Compose		Multistage build
Despliegue	Railway		+ GitHub Actions Continuous Integration / Continuous D
Tests unitarios	PHPUnit		13.x, tests paralelos
Tests E2E	Playwright		TypeScript
Análisis estático	PHPStan + Psalm		Nivel 5 + baseline
Email	Mailpit (dev)		SMTP local
Facturas PDF	Dompdf		Generación asíncrona

1.6. Metodología de desarrollo

El desarrollo siguió una metodología ágil inspirada en **Scrum**, adaptada al contexto de un equipo unipersonal. Se utilizó Trello como tablero Kanban con las columnas *Backlog*, *En progreso*, *Revisión* y *Hecho*.

El control de versiones se gestionó con Git siguiendo el flujo **Git Flow**:

- `main` —código en producción (Railway)
- `develop` —integración continua

- *feature/nombre* —desarrollo de funcionalidades
- *hotfix/nombre* —correcciones urgentes

Cada *pull request* requería superar el *quality gate* completo: PHPStan nivel 5, Psalm, suite PHPUnit y verificación de estilo PSR-12.

Capítulo 2

Desarrollo Técnico

2.1. Arquitectura del sistema

Komorebi Café implementa el patrón MVC desde cero, sin delegar en ningún framework de aplicación externo. El objetivo es demostrar comprensión profunda de los mecanismos subyacentes: enrutamiento, inyección de dependencias, ciclo de petición-respuesta y gestión de middleware.

2.1.1. Capas principales

`public/index.php` —Front controller

Único punto de entrada. Lee variables de entorno, inicializa el contenedor de dependencias y pasa la petición al router.

`bootstrap/container.php` —Contenedor IoC

Registra todos los *Service Providers*. Ciclo `register→boot`. Singletons enlazados por interfaz.

`app/routes.php` —Definición de rutas

250 rutas agrupadas por dominio con prefijos y middleware declarados de forma explícita.

`app/Core/` —Framework interno

Router PSR-7/15, Contenedor IoC, Vista (auto-escape + Raw), Base de datos (PDO), Caché (Redis + local), Cola (Redis), Logger (stdout 12-Factor).

`app/Http/Controllers/` —Controladores

Agrupados en: `Admin/`, `Api/V1/`, `Auth/`, `Kitchen/`, `Keeper/`, `Manager/`, `Public/`, `Reception/`, `Shared/`, `Supervisor/`, `User/`.

`app/Services/` —Lógica de negocio

53 servicios. Todos devuelven `Result::ok()/fail()` —nunca lanzan excepciones para fallos esperados.

app/Repositories/ —Capa de datos

29 repositorios. Extienden `AbstractRepository` con PDO directo.

app/Events/ + Listeners/ —Eventos asíncronos

PSR-14 con Symfony EventDispatcher. Procesados por workers Redis.

migrations/ —Esquema de base de datos

19 archivos SQL numerados, aplicables mediante `make db-migrate`.

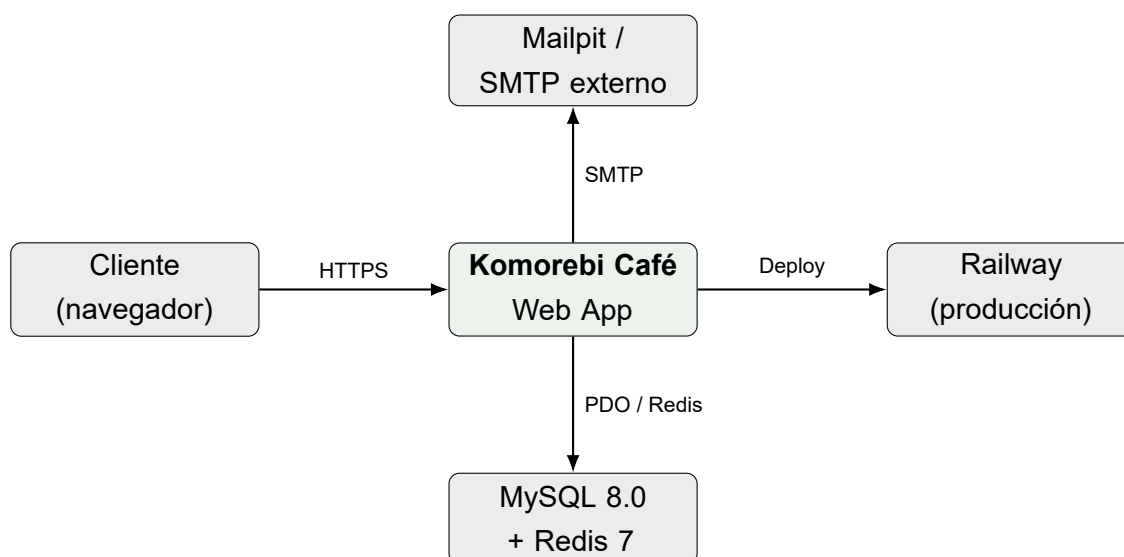
2.1.2. Diagrama de contexto C4 (nivel 1)

Figura 2.1: Diagrama de contexto C4 —nivel sistema

2.2. Ciclo de petición HTTP y pipeline de middleware

Cada petición HTTP atraviesa un pipeline de quince middlewares PSR-15 antes de llegar al controlador correspondiente. La ?? muestra el orden de ejecución.

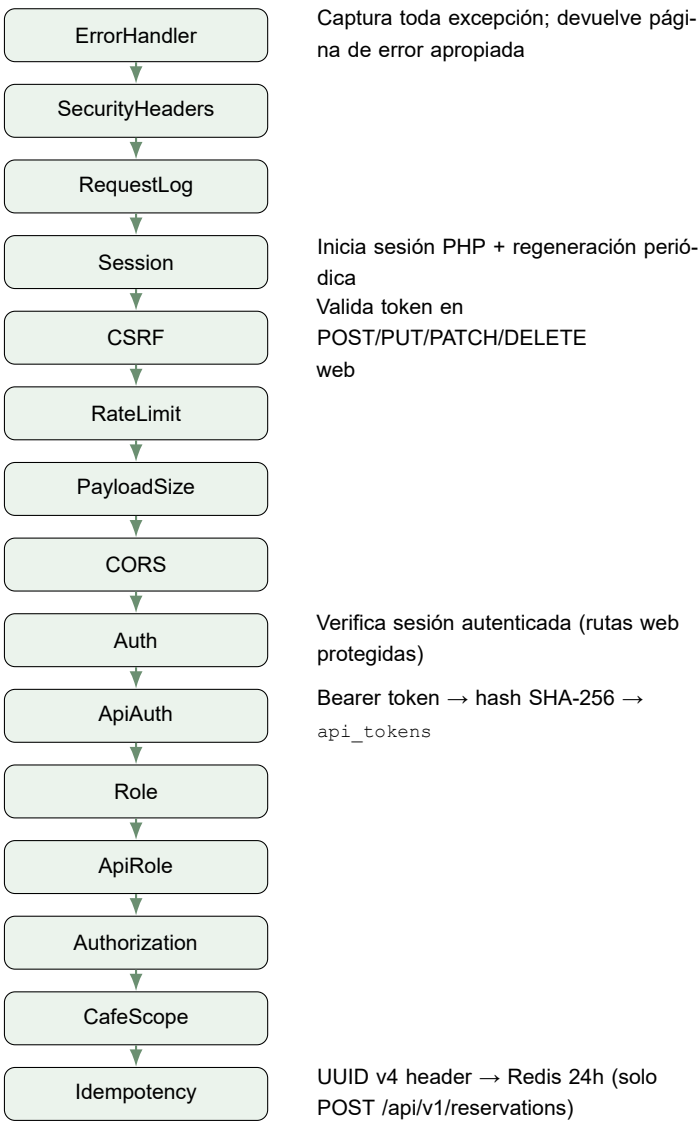


Figura 2.2: Pipeline de 15 middlewares PSR-15

La ?? describe brevemente cada middleware.

Tabla 2.1: Catálogo de middlewares PSR-15

Middleware	Responsabilidad
ErrorHandlerMiddleware	Envuelve toda la cadena; captura <code>Throwable</code> y devuelve respuesta de error apropiada (HTML 4xx/5xx o JSON según el tipo de ruta).

Middleware Responsabilidad	
SecurityHeadersMiddleware	Añade cabeceras de seguridad: X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, Referrer-Policy.
RequestLogMiddleware	Registra método, URI, IP, agente y tiempo de respuesta en stdout (12-Factor XI).
SessionMiddleware	Inicia la sesión PHP, gestiona regeneración periódica del identificador y establece atributos de sesión en el objeto Request.
CsrfMiddleware	Verifica el token CSRF en todas las rutas mutantes (POST/PUT/PATCH/DELETE) web. Rechaza con 419 si el token falta o no coincide.
RateLimitMiddleware	Limitación de velocidad por clave (bucket Redis): login, registration, password_reset. Responde 429 al exceder.
PayloadSizeMiddleware	Rechaza cuerpos de petición superiores a 10 MB para prevenir ataques de agotamiento de memoria.
CorsMiddleware	Gestiona cabeceras CORS y respuestas OPTIONS preflight para la API.
AuthMiddleware	Requiere sesión autenticada. Redirige a /login si no está logado.
ApiAuthMiddleware	Extrae el Bearer token, aplica SHA-256, consulta api_tokens y autentica la petición de forma stateless.
RoleMiddleware	Verifica que el rol del usuario en sesión coincide con el rol requerido por la ruta web.
ApiRoleMiddleware	Igual que RoleMiddleware pero para peticiones API autenticadas con Bearer token.

Middleware Responsabilidad

<code>AuthorizationMiddleware</code>	Aplica políticas de autorización de grano fino (p.ej. solo el propietario puede modificar sus datos).
<code>CafeScopeMiddleware</code>	Restringe el acceso de managers y supervisors a los datos del café al que están asignados.
<code>IdempotencyMiddleware</code>	Verifica el header <code>Idempotency-Key</code> (UUID v4); devuelve la respuesta cacheada (Redis, TTL 24 h) si existe, o ejecuta y almacena el resultado.

2.3. Flujos de autenticación

El sistema implementa tres mecanismos de autenticación diferenciados según el tipo de cliente.

2.3.1. Login web con sesión

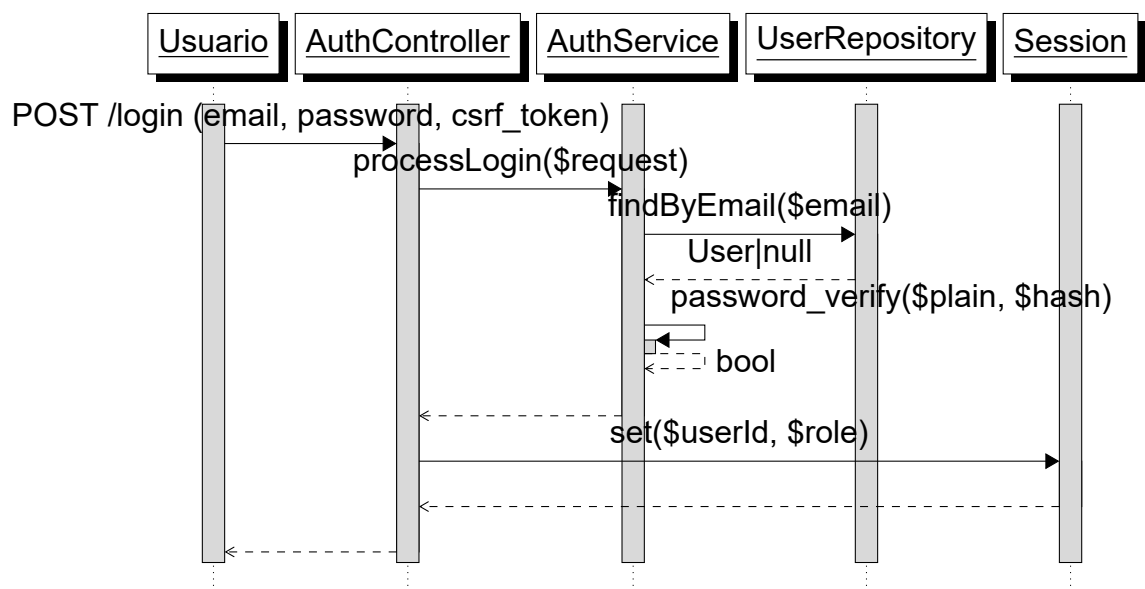


Figura 2.3: Diagrama de secuencia: login web